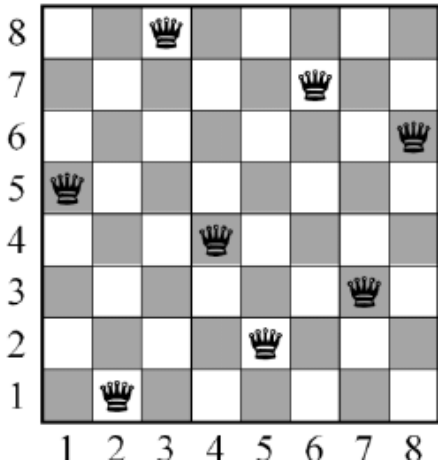
Constraint Satisfaction Problems

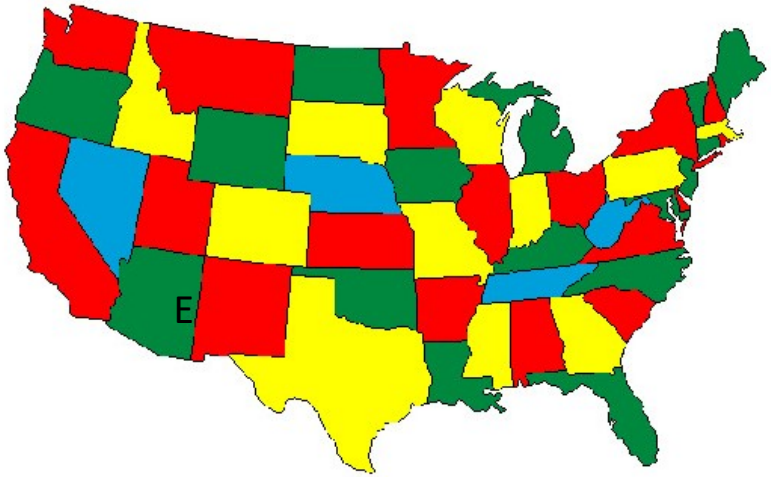
Search a solution with many constraints/restrictions:

Constraint Satisfaction Problem

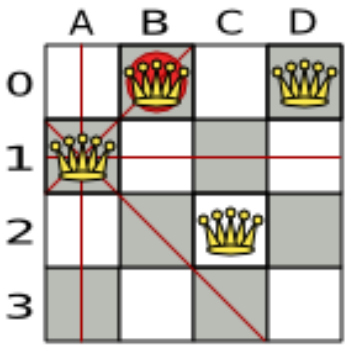


Each piece much match perfectly with its neighbors' shape

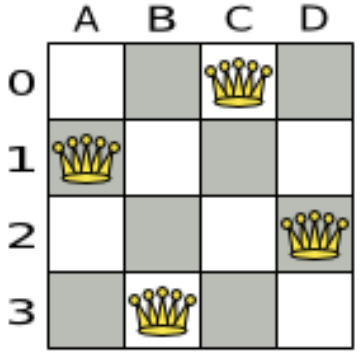




4-queen puzzle

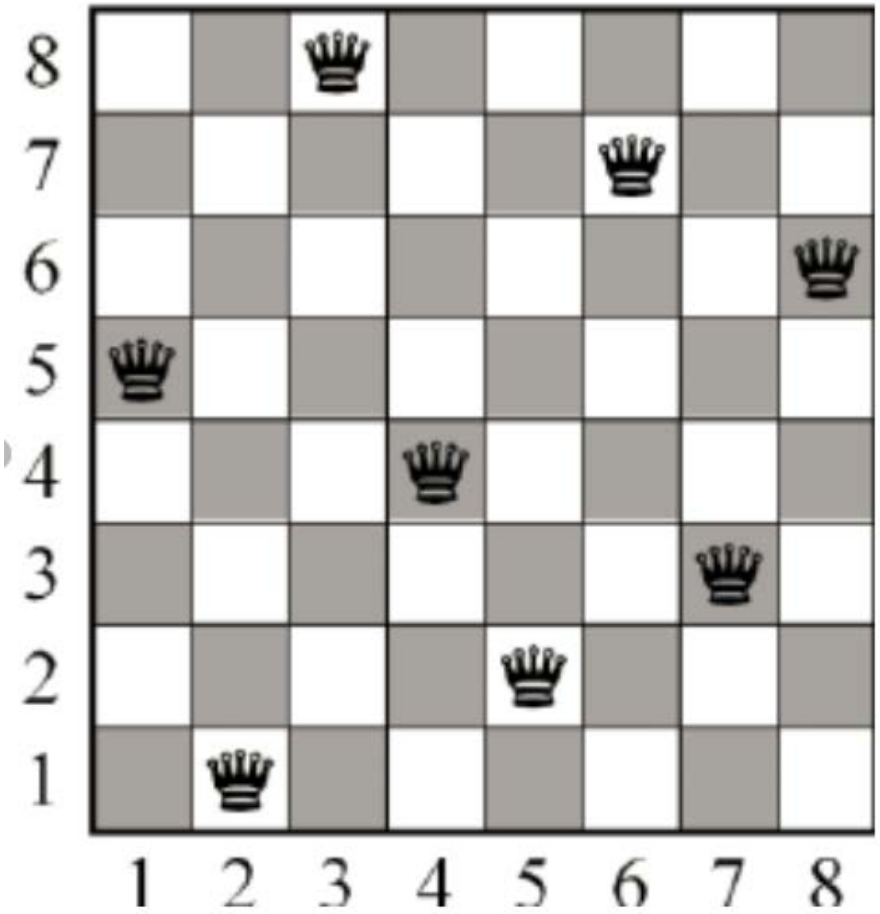


A queen can move horizontally, vertically, or diagonally.



A conflict free solution

8-Queen Puzzle

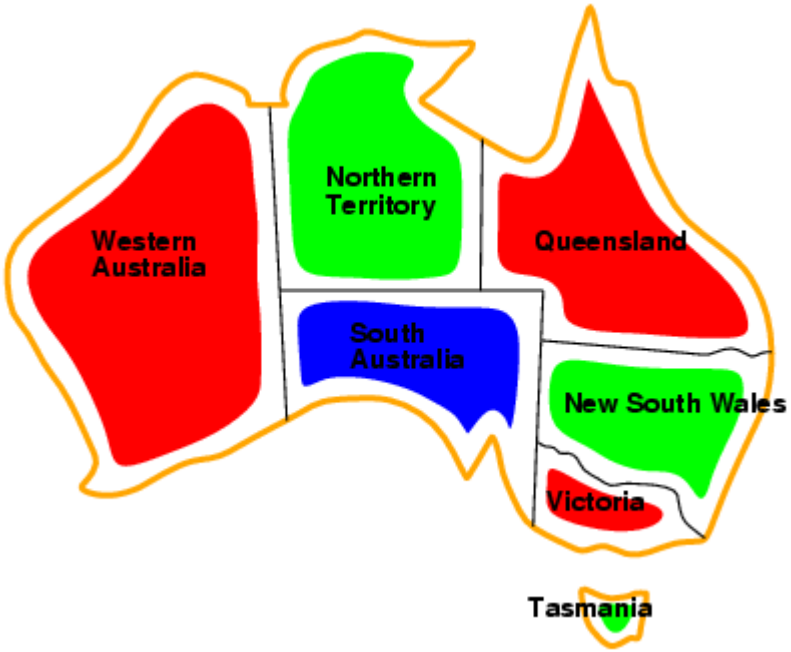


For any country map or world map, 4 colors are sufficient.



Map coloring problem





Real-world CSPs

Assignment problems
    e.g., who teaches what class

Timetabling problems
    e.g., which class is offered when and where?

Hardware configuration

Spreadsheets

Transportation scheduling

Factory scheduling

Floorplanning

Notice that many real-world problems involve real-valued variables

CSP computational algorithms

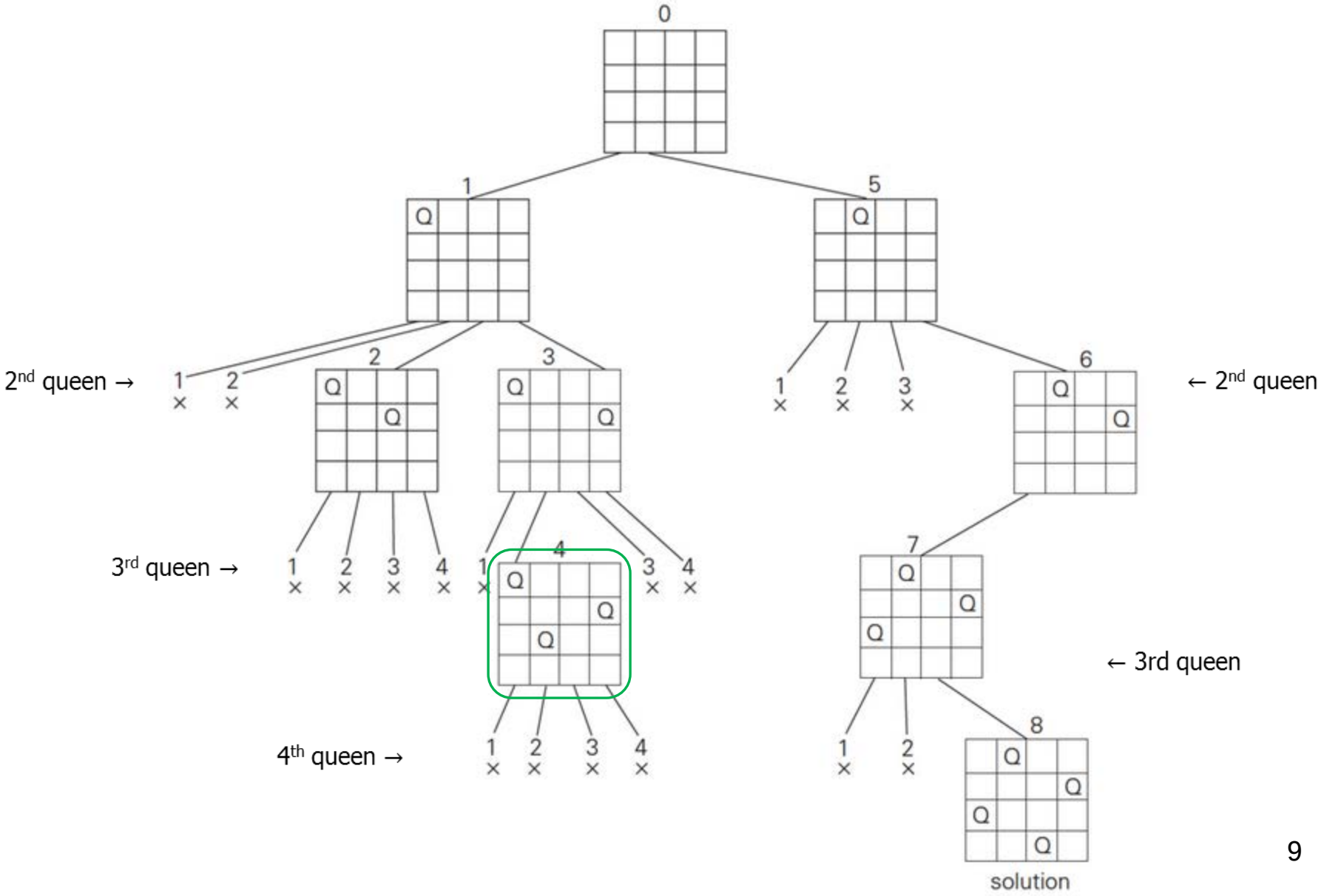
1. Main algorithm: Backtracking algorithm.

   Do exhaustive search to find one solution.

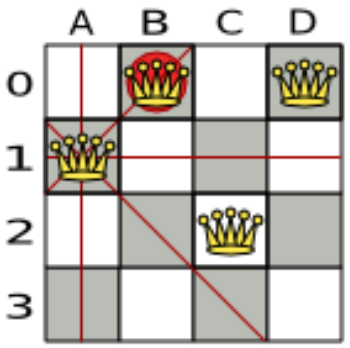
2. Use three heuristics to improve/speedup the backtracking search. Use arc constancy for better understanding and efficient solutions.

3. Use min-conflict to quickly find a solution

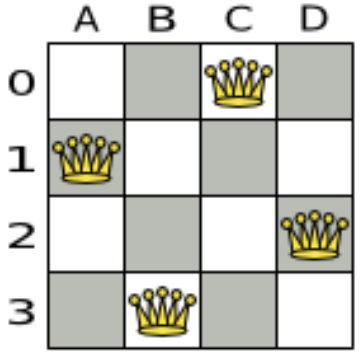
- Use backtracking algorithm to solve the 4-queen problem



4-queen puzzle

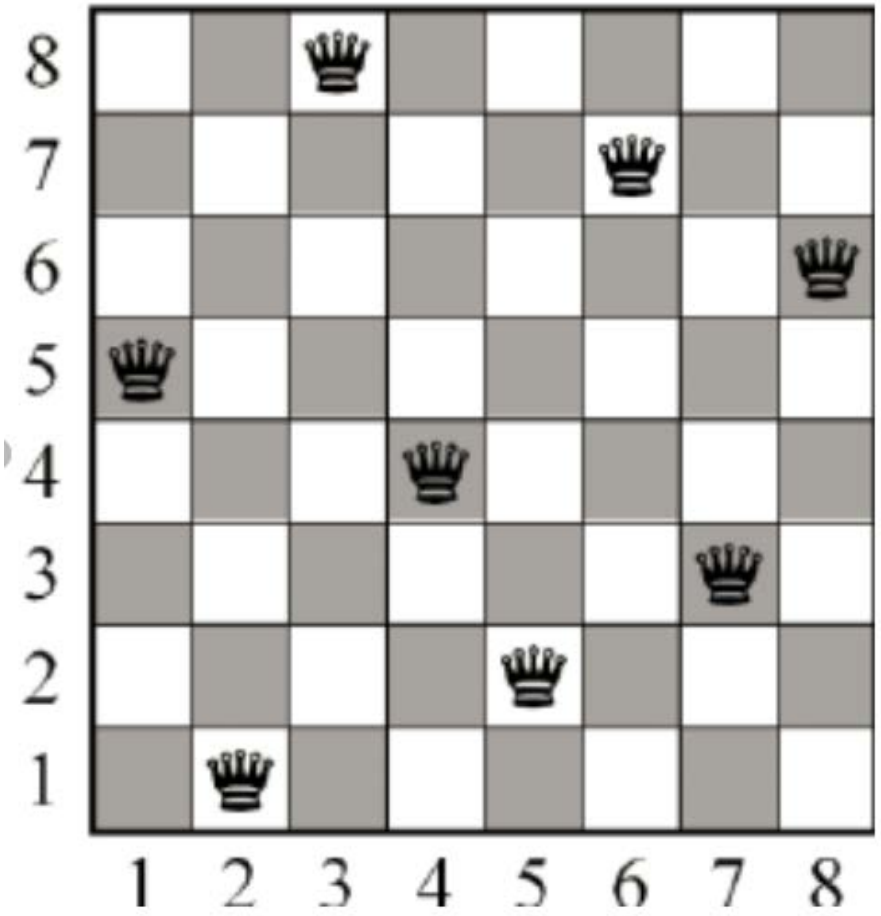


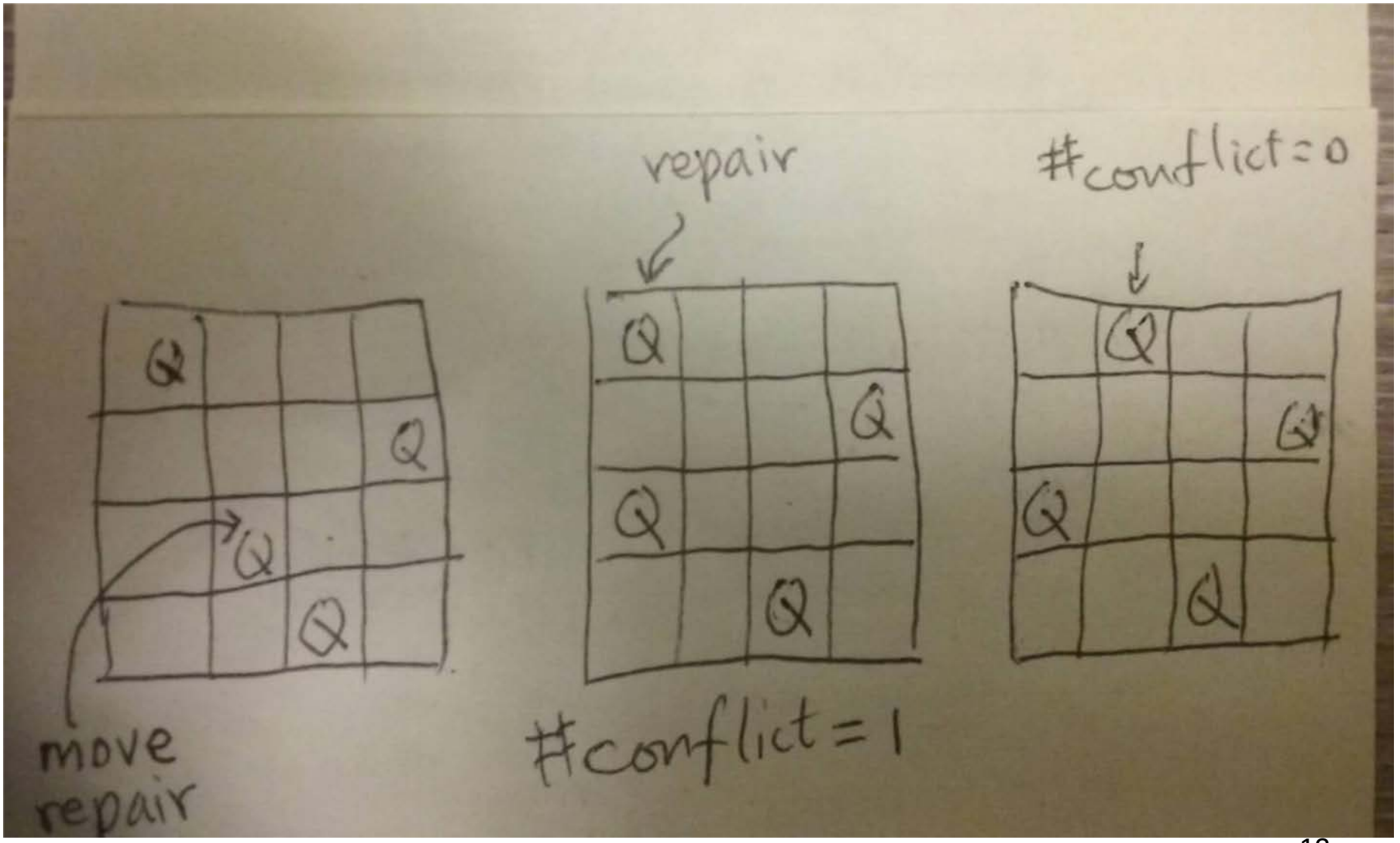
A queen can move horizontally, vertically, or diagonally.



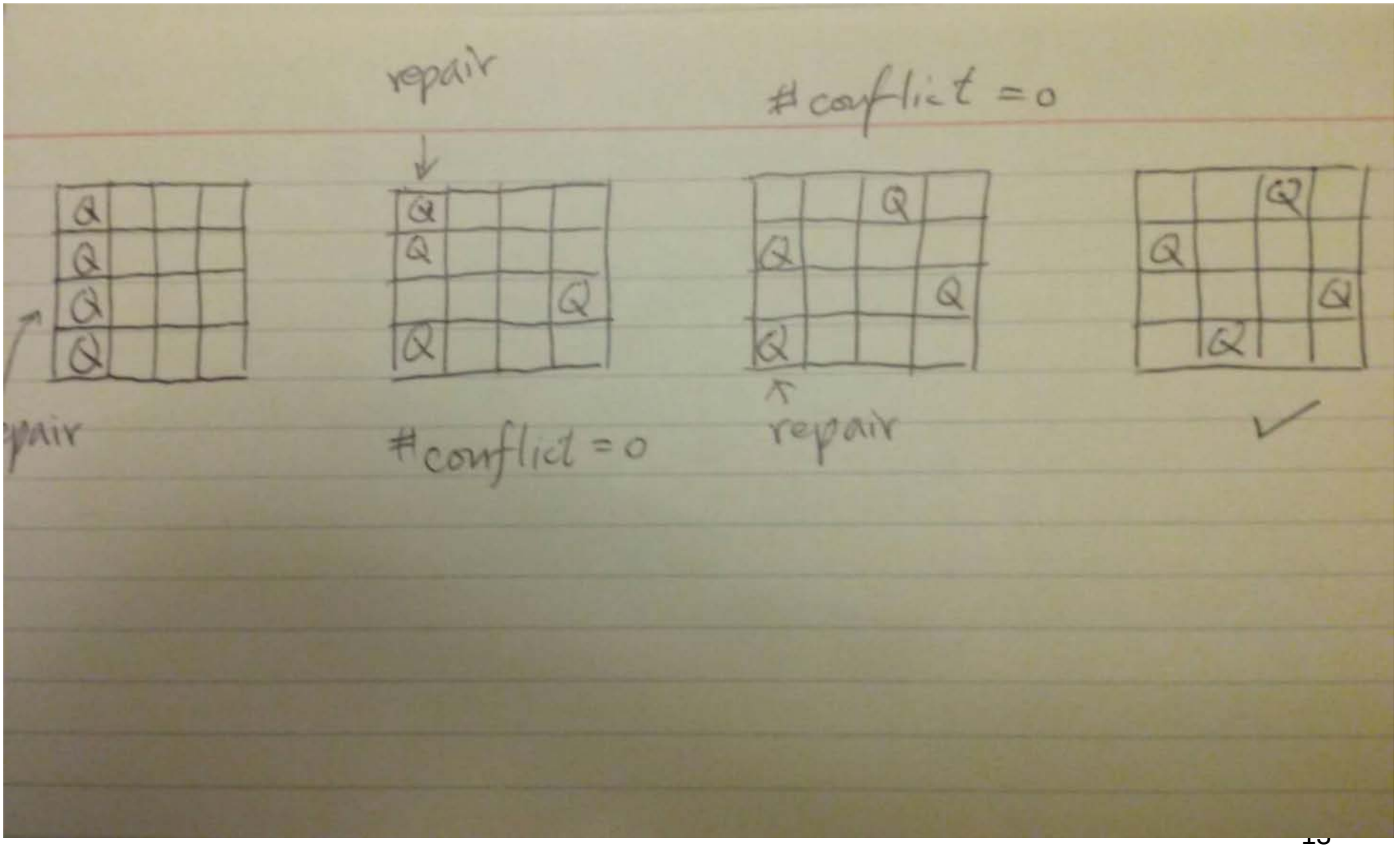
A conflict free solution

8-Queen Puzzle




repair
#conflict=0
move
repair
#conflict=1

Solve 4-queen problem using
minimum conflict repair algorithm



Constraint satisfaction problems (CSPs)

- CSP:
  - state is defined by variables X_i with values from domain D_i
  - goal test is a set of constraints specifying allowable combinations of values for subsets of variables

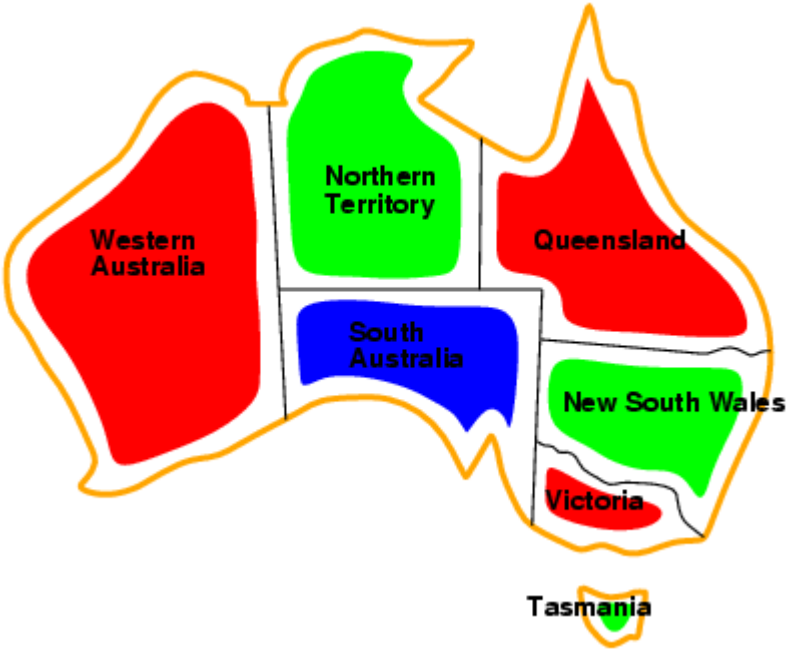
- Allows useful general-purpose algorithms with more power than standard search algorithms

Example: Map-Coloring



- Variables *WA, NT, Q, NSW, V, SA, T*
- Domains D_i = {red,green,blue}
- Constraints: adjacent regions must have different colors
- e.g., WA ≠ NT

Example: Map-Coloring

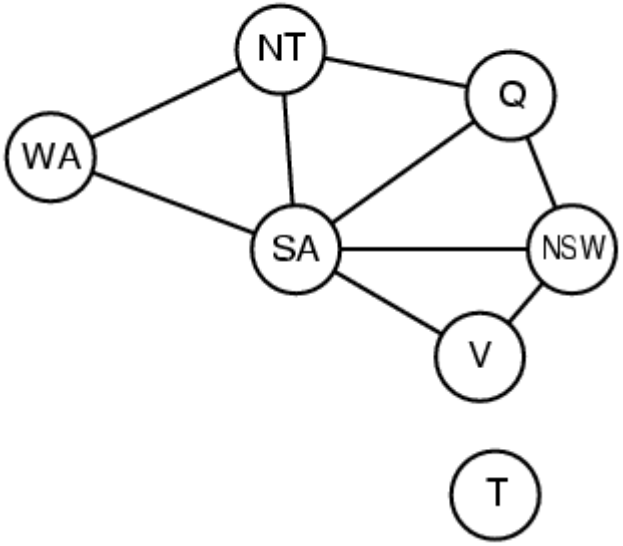


- Solutions are complete and consistent assignments, e.g., WA = red, NT = green,Q = red,NSW = green,V = red,SA = blue,T = green

Constraint graph

- Binary CSP: each constraint relates two variables
- Constraint graph: nodes are variables, arcs are constraints





Varieties of CSPs

- Discrete variables
  - finite domains:
    - n variables, domain size $d \rightarrow O(d^n)$ complete assignments
    - e.g., 3-SAT (NP-complete)
  - infinite domains:
    - integers, strings, etc.
    - e.g., job scheduling, variables are start/end days for each job:
      $StartJob_1 + 5 \leq StartJob_3$

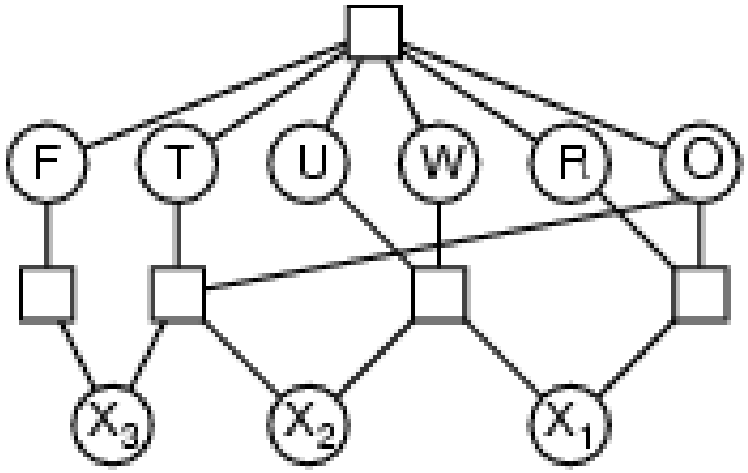
- Continuous variables
  - linear objective & constraints solvable in polynomial time by linear programming
  - There are very good, off-the-shelves, methods for convex optimization problems.

Varieties of constraints

- Unary constraints involve a single variable,
  - e.g., SA ≠ green
- Binary constraints involve pairs of variables,
  - e.g., SA ≠ WA
- Higher-order constraints involve 3 or more variables,
  - e.g., SA ≠ WA ≠ NT

Crypto-arithmetic Puzzle

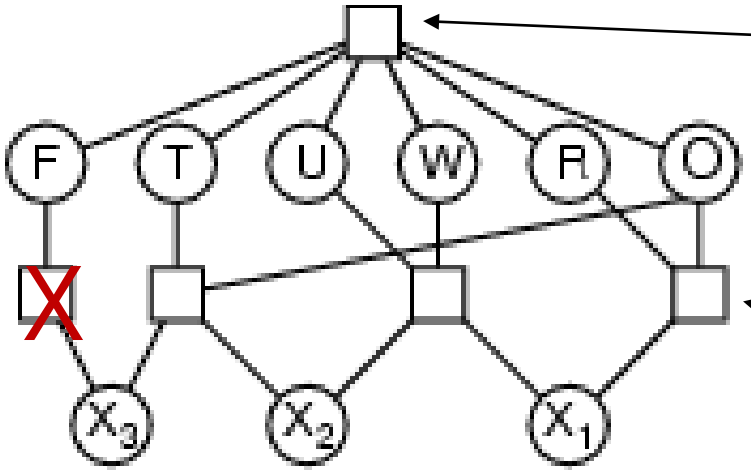
$$\begin{array}{cccc} & T & W & O \\ + & T & W & O \\ \hline F & O & U & R \end{array}$$



- Variables: *F T U W R O* $X_1\ X_2\ X_3$
- Domains: {*0,1,2,3,4,5,6,7,8,9*} {0,1}
- Constraints: *Alldiff (F,T,U,W,R,O)*
  - $O + O = R + 10 \cdot X_1$
  - $X_1 + W + W = U + 10 \cdot X_2$
  - $X_2 + T + T = O + 10 \cdot X_3$
  - $X_3 = F,\ T \neq 0,\ F \neq 0$

Crypto-arithmetic Puzzle RN p.202

```
  T W O
+ T W O
-------
F O U R
```



Hypergraph edge

$X_1 \neq R \neq O \neq X_1$

- Variables: *F T U W R O* $X_1\ X_2\ X_3$
- Domains: {*0,1,2,3,4,5,6,7,8,9*} {0,1}
- Constraints: Alldiff *(F,T,U,W,R,O)* variables have different values
  - $O + O = R + 10 \cdot X_1$
  - $X_1 + W + W = U + 10 \cdot X_2$
  - $X_2 + T + T = O + 10 \cdot X_3$
  - $X_3 = F,\ T \neq 0,\ F \neq 0$

$X_1 \neq X_2$

$X_3 \neq X_2$

Correct?

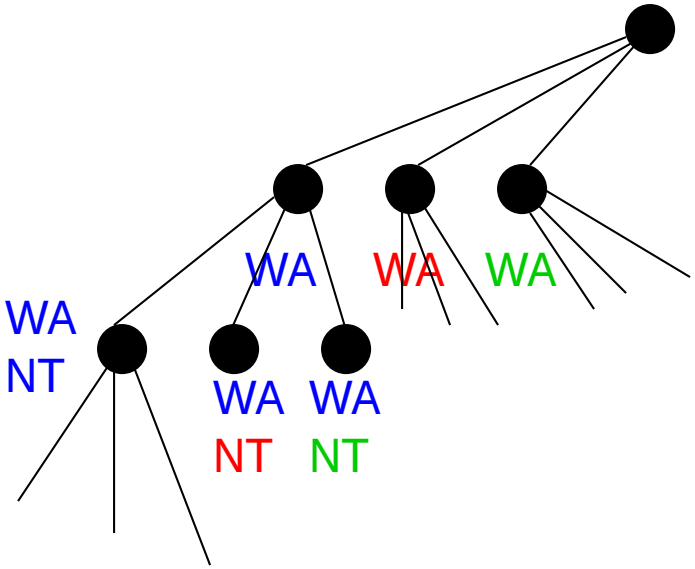
Homework for Ph.D. students: Solve this problem using backtracking

Backtracking (Depth-First) search

- Special property of CSPs: They are commutative:
  This means: the order in which we assign variables does not matter.

NT WA = WA NT

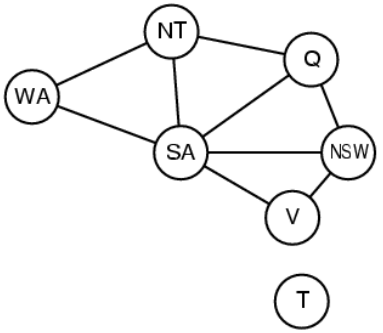
- Search tree: First order variables, then assign them values one-by-one.



D

D^2

D^N



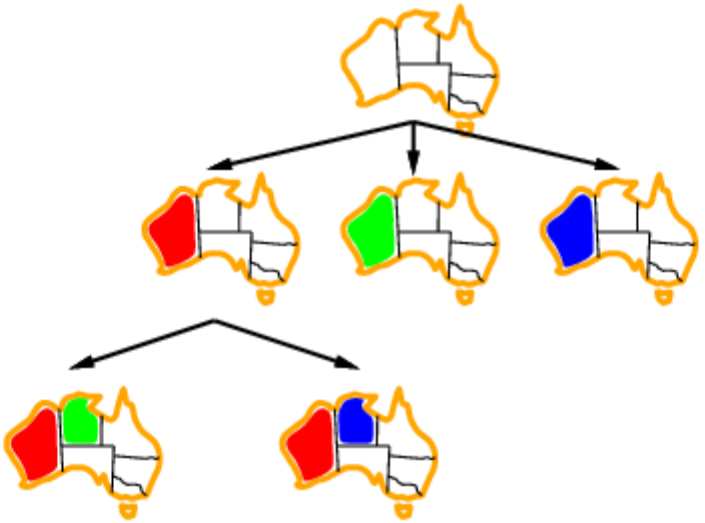
Backtracking example



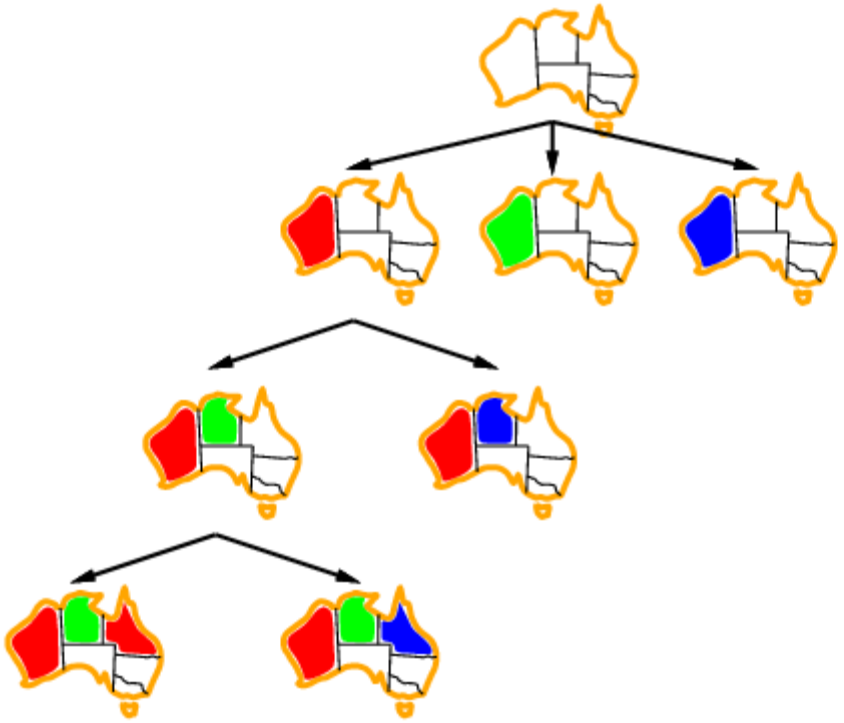
Backtracking example



Backtracking example



Backtracking example



Improving backtracking efficiency

- The heuristics/tricks give huge gains in speed:

1 Which variable should be assigned next?

2 In what order should its values be tried?

3 Can we detect wrong search tracks early on?

4. Arc consistency, better understanding constraints

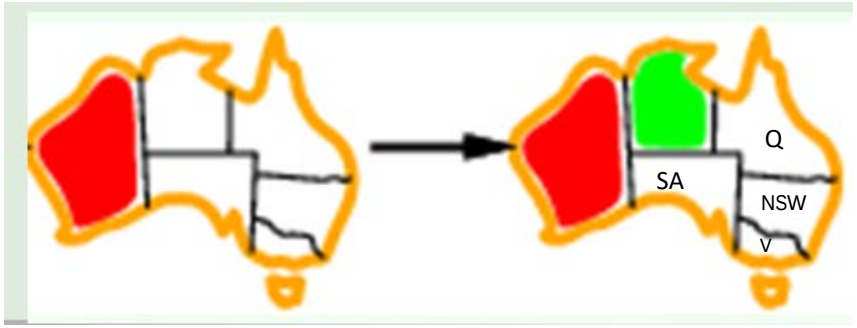
We discuss heuristics for these questions below

Heuristics 1: Pick next variable/node

- Pick the most constrained variable next:

  i.e., pick the variable with the minimum remaining values(MRV)





Question: Now that we colored WA. Next, pick which state/variable to color?

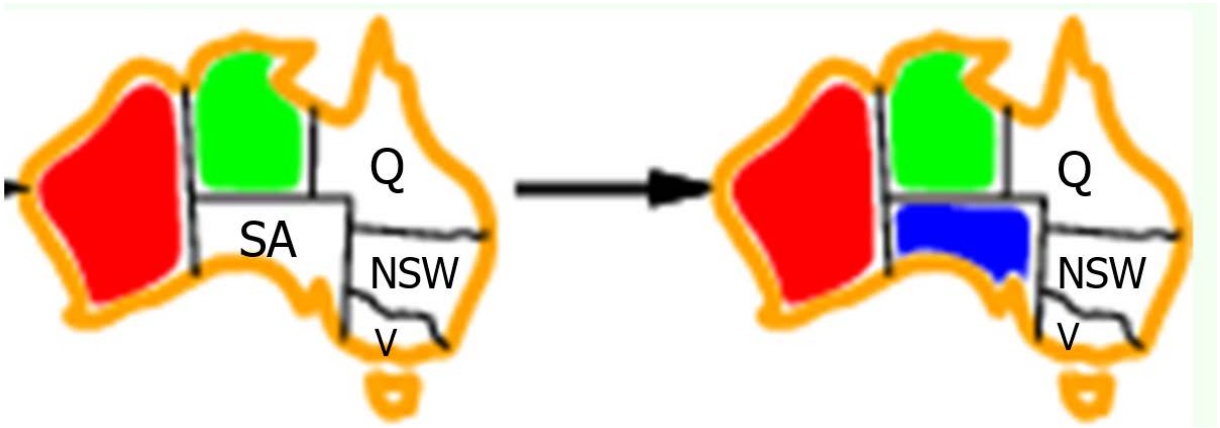
Analysis:
NT has 2 choices of color. SA has two choices. The rest states each has 3 choices.

V

The MRV heuristic choose either NT or SA. So we choose NT (for simplicity) to color.

Here we can choose blue or green. We choose green for simplicity.

Heuristics 1b: Pick next variable



1. WA and NT are colored. Which variable to color next?

2. Now SA has 1 choice of color. Q has 2 choices. The rest, each has 3 choices.

3. The heuristic choose SA. So we color SA with blue.

Heuristics 1c: Pick next variable





1. Q, NSW, V are not colored. Which variable to color next?

2. Now Q has 1 choice of color. NSW and V, each has 2 choices.

3. The heuristic choose Q. So we color Q with red.

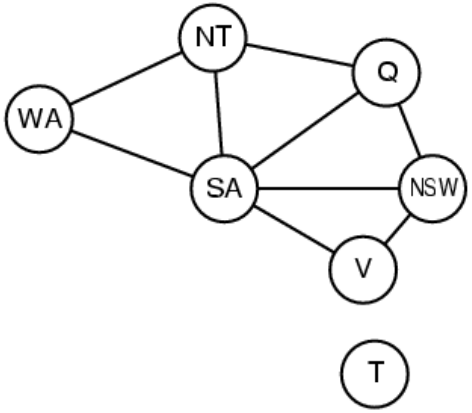
Students should work out the rest of colorings using the RMV heuristic.

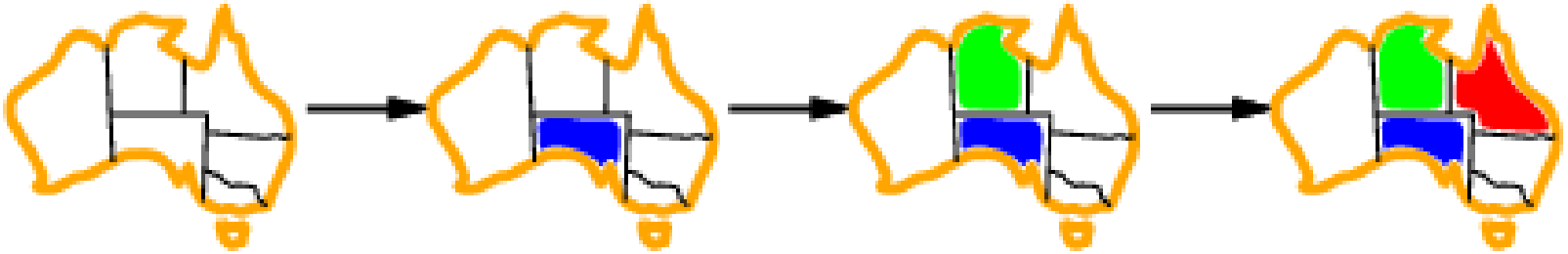
Degree of a node =number of connected edges

Heuristics 1d: Pick next variable

Further development: degree heuristic

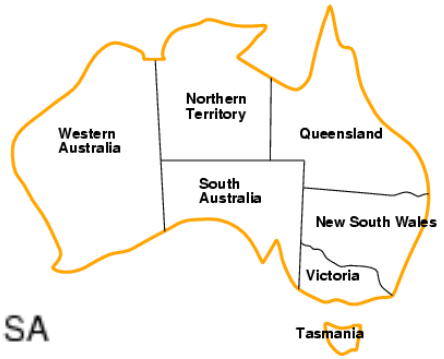
- Pick the most constrained variables
  - choose the variable with the most constraints on remaining variables (most edges in graph)
  - The degree of nodes is a good approximation
  - Among all remaining nodes, pick the one with largest degree.

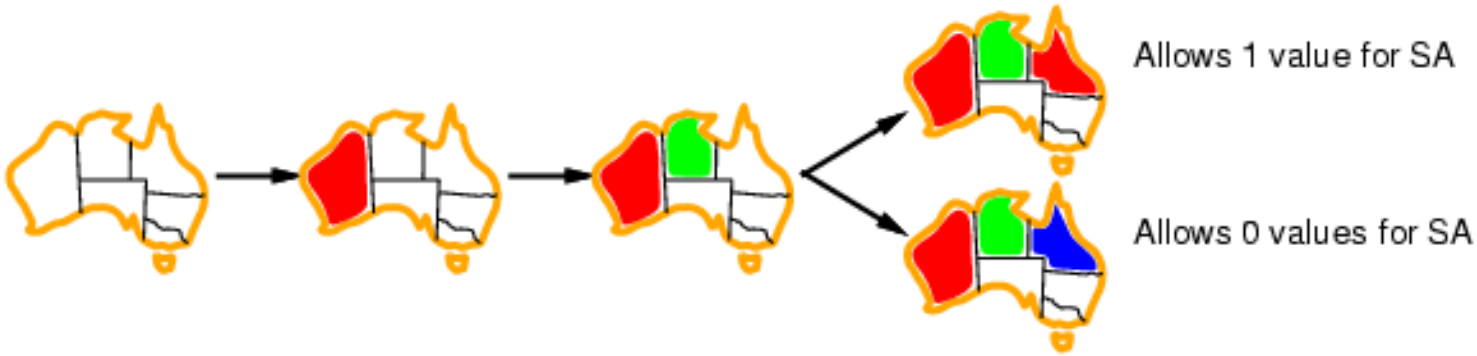




Heuristics 2: Pick which color: Pick the color that least constraining values (LCV) on adjacent variables

- *Given a variable*, choose the value that gives least constraints on the remaining variables





- Leaves maximal flexibility for coloring remaining variables
- Combining these heuristics makes 1000 queens feasible

Rationale for 2 heuristics

- In all cases we want to enter the most promising branch, but we also want to detect dead-end search tract early on.

- 1st heuristic: the variable that is most likely to cause failure in a branch is assigned first. The variable must be assigned at some point; so if it is doomed to fail, we'd better found out soon.

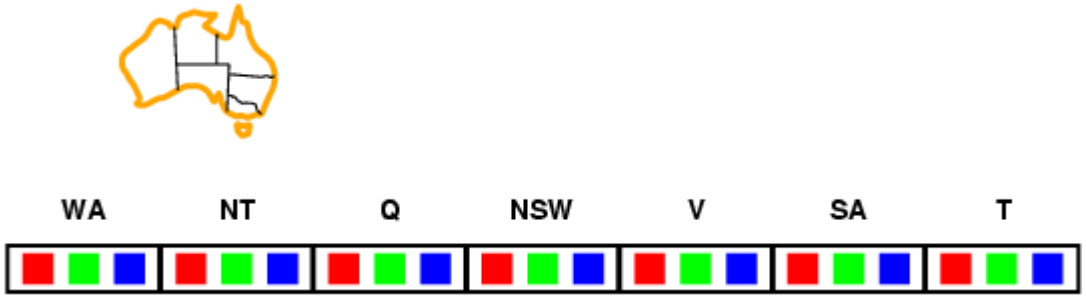
- 2nd heuristic: tries to avoid failure by assigning colors that leave maximal flexibility for the remaining variables, thus more likely to succeed.

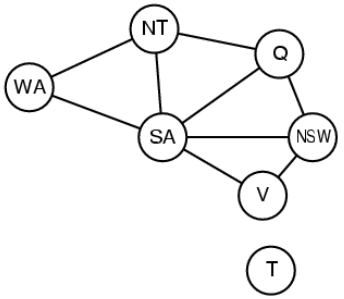
3rd Heuristics:

Can we detect dead-end search track early on?

→ forward checking

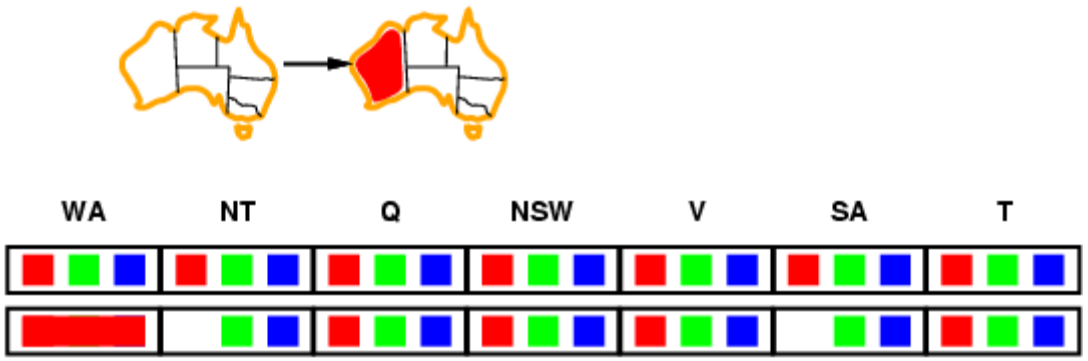
- Idea:
  - Keep track of remaining legal values for unassigned variables that are connected to current variable.
  - Terminate search when any variable has no legal values

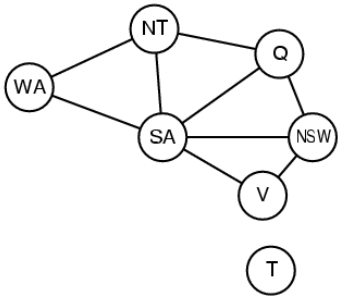




Forward checking

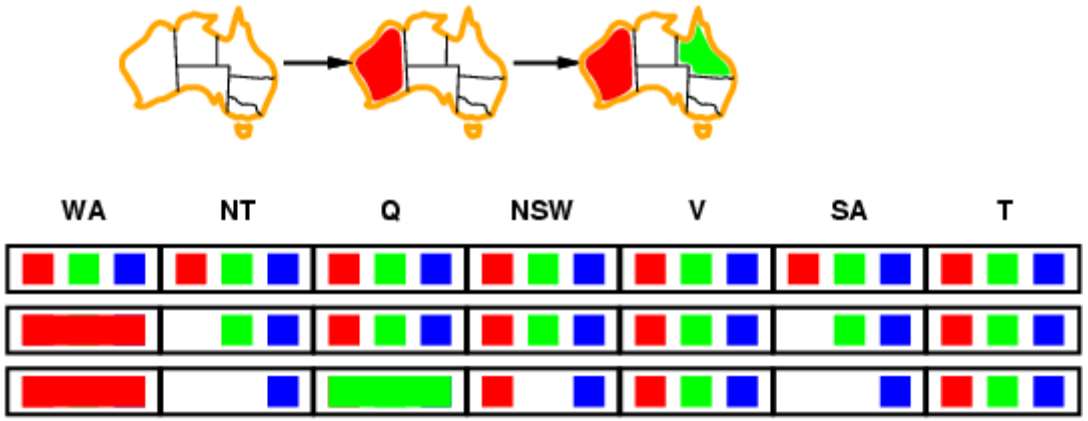
- Idea:
  - Keep track of remaining legal values for unassigned variables
  - Terminate search when any variable has no legal values

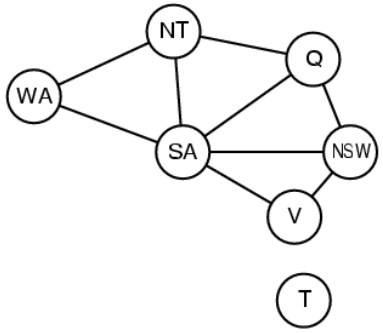




Forward checking

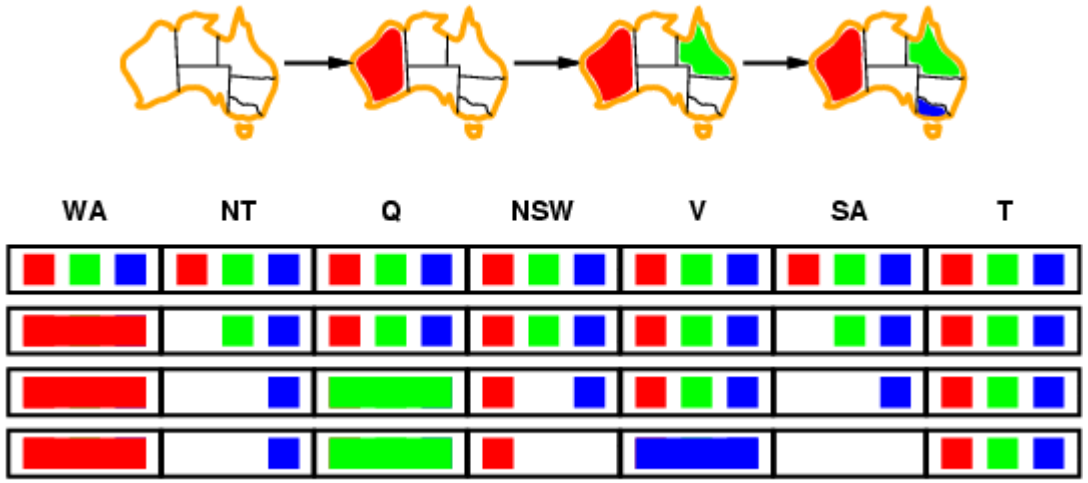
- Idea:
  - Keep track of remaining legal values for unassigned variables
  - Terminate search when any variable has no legal values

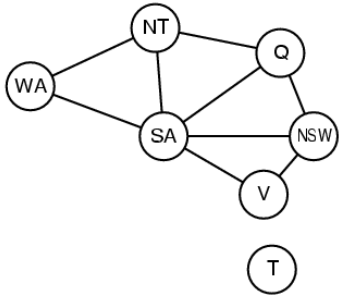




Forward checking

- Idea:
  - Keep track of remaining legal values for unassigned variables
  - Terminate search when any variable has no legal values because SA has no values that fits.



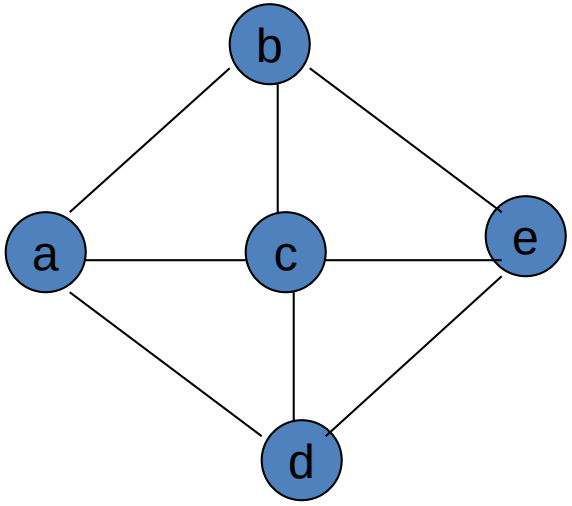


Homework: CSP1.

Consider the constraint graph on the right.
The domain for every variable is [1,2,3,4], i.e., each node is assigned one of 1,2,3,4.
There are 2 unary constraints:
- variable “a” cannot take values 3 and 4.
- variable “b” cannot take value 4.

There are 8 binary constraints stating that variables connected by an edge cannot have the same value.

Find a solution for this CSP by using the following heuristics: minimum value heuristic, degree heuristic, forward checking. Explain each step of your answer.

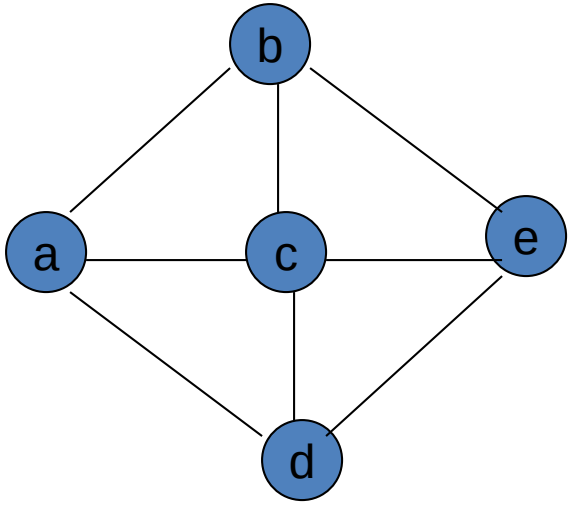


CONSTRAINT GRAPH

2) Consider the constraint graph on the right.
The domain for every variable is [1,2,3,4].
There are 2 unary constraints:
- variable “a” cannot take values 3 and 4.
- variable “b” cannot take value 4.
There are 8 binary constraints stating that variables connected by an edge cannot have the same value.

Find a solution for this CSP by using the following heuristics: minimum value heuristic, degree heuristic, forward checking. Explain each step of your answer.

MVH→	*a=1 (for example)*
FC+MVH→	*b=2*
FC+MVH→	*c=3*
FC+MVH→	*d=4*
FC→	*e=1*



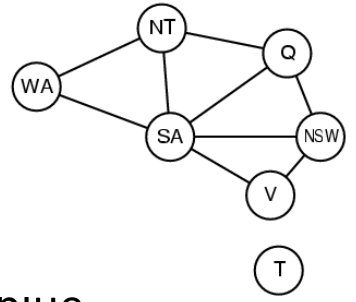
CONSTRAINT GRAPH

Constraint propagation (RN: Chap 6.2)

- Forward checking only checks consistency between assigned and non-assigned states.
- How about constraints between two unassigned states? This leads to a new and powerful constraint propagation algorithm (CPA)







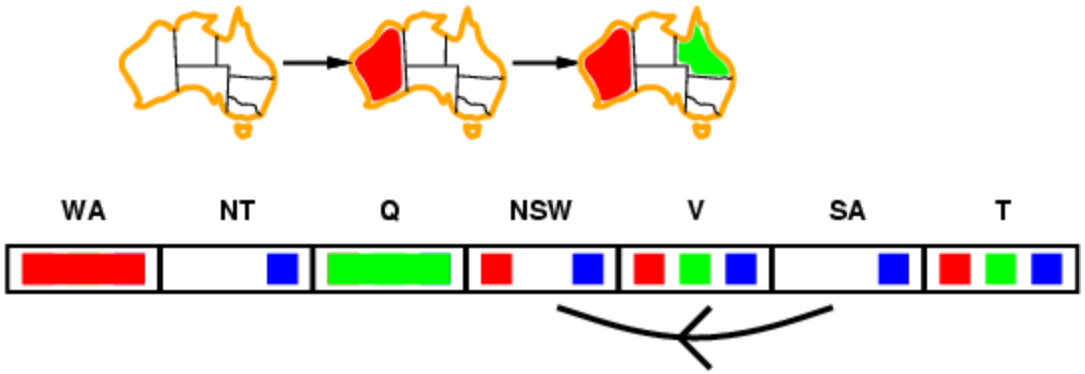
e choice: blue.

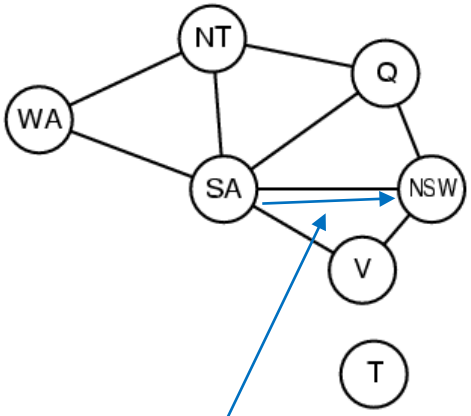
- But NT and SA cannot both be blue! Dead end!
- Constraints propagated so that two unassigned variables get more local constraints (in this case, unsolvable constraints).

Arc consistency

- Simplest form of constraint propagation is arc consistent
- Defination: $X \rightarrow Y$ is consistent

  iff for every remaining value of *X*, there is some allowed value for *Y*





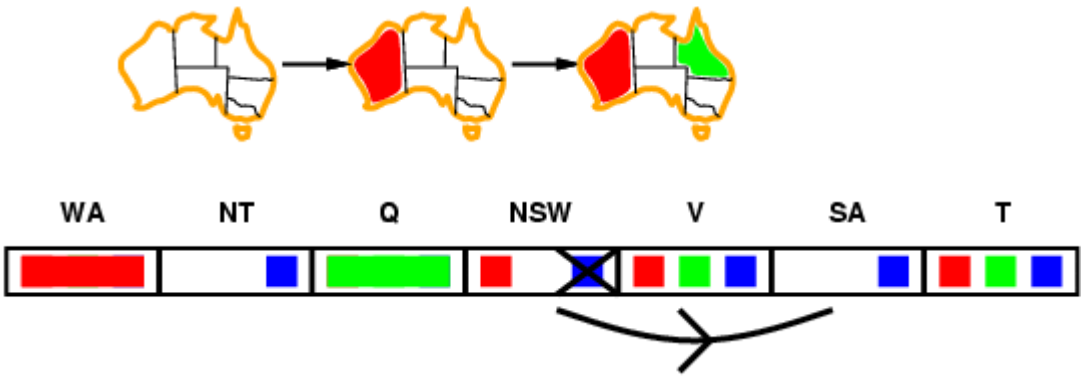
At current stage, arc SA → NSW is consistent

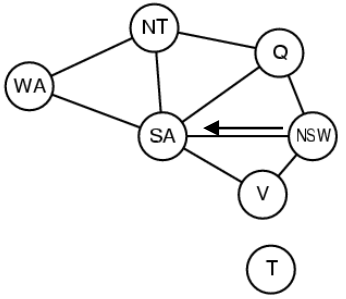
Constraint propagation propagates arc consistency on the graph.

Arc consistency

- Simplest form of propagation makes each arc consistent
- *X* → *Y* is consistent iff (if and only if)

  for every remaing value of *X*. there is some allowed value for Y



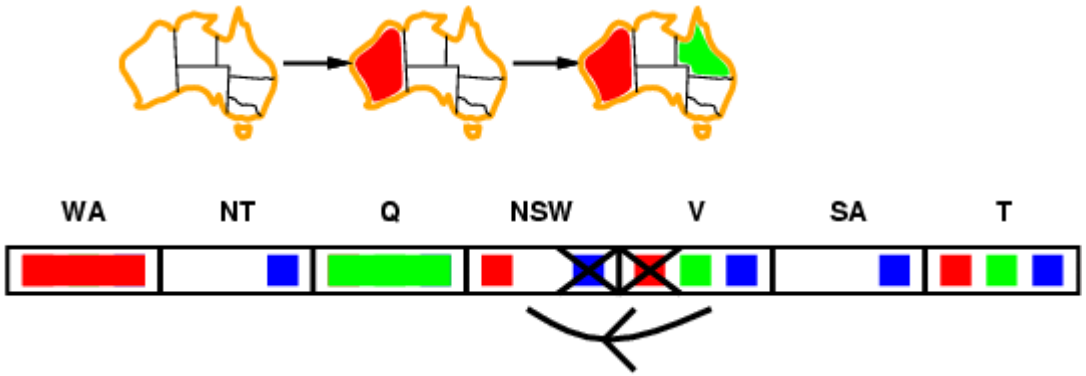


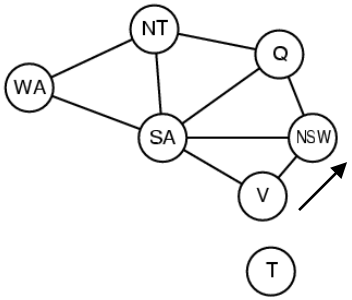
At current stage, arc NSW → SA is inconsistent,
Because if NSW picks blue, there is no choice for SA.

Color propagation algorithm (CPA)
CPA remove blue from source. → consistent arc.

Arc consistency

- Simplest form of propagation makes each arc consistent
- $X \rightarrow Y$ is consistent iff
  for every remaining value of X, there is some allowed value for Y

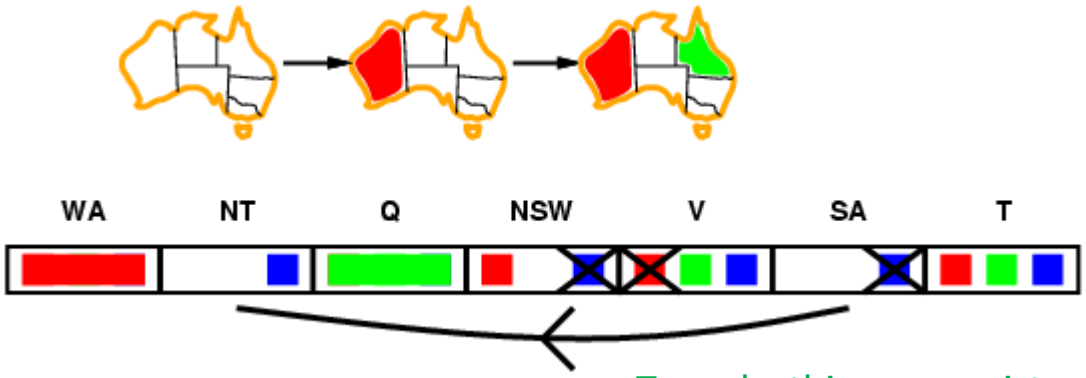


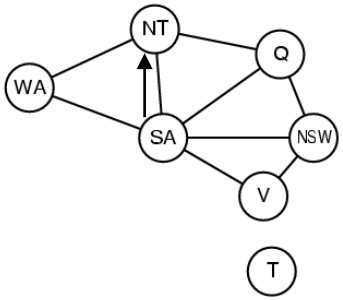


this arc just became inconsistent

i.e. incoming arcs can become inconsistent again
(outgoing arcs will stay consistent).

Arc consistency algorithm





To make this arc consistent, we need to delete blue from SA's domain. Thus SA cannot be assign a color. This route of coloring is a failure.

- If *X* loses a value, neighbors of *X* need to be rechecked
- Arc consistency detects failure earlier than forward checking
- Can be run as a preprocessor or after each assignment

- Time complexity: $O(n^2d^3)$

arcs

d^2 for checking, each node can be checked d times at most

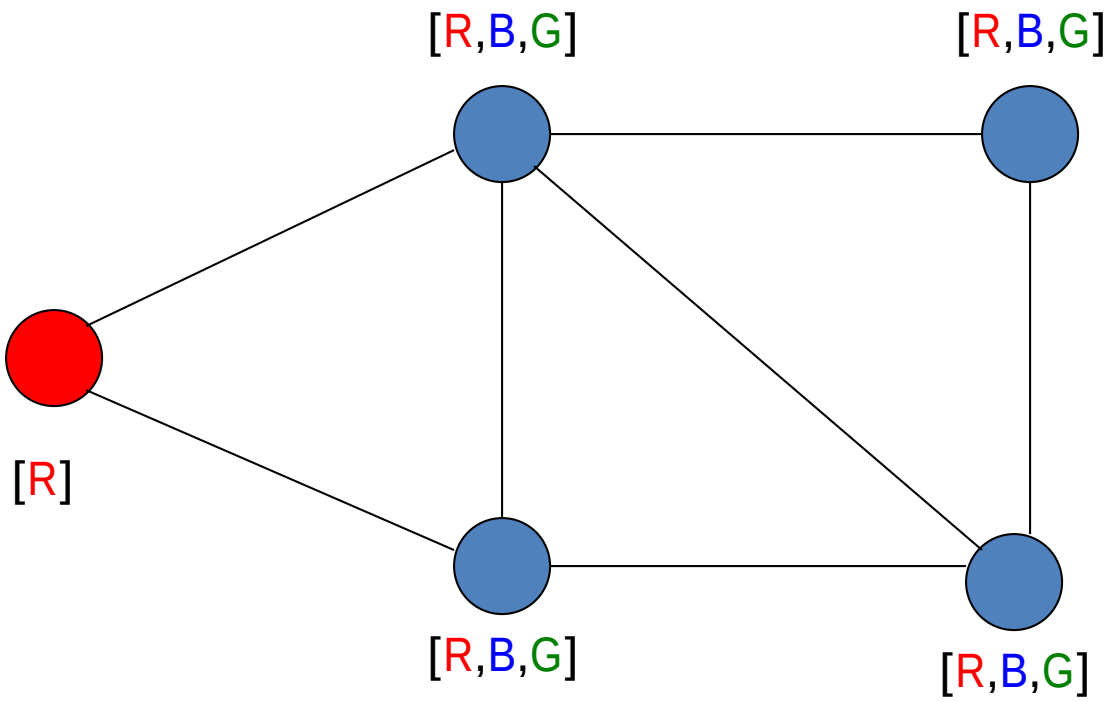
Arc Consistency Algorithm

- This is a constraint propagation algorithm. It's like sending messages to neighbors on the graph! How do we schedule these messages?
- Every time a domain changes, all incoming messages need to be re-send. Repeat until convergence → no message will change any domains.
- Since we only remove values from domains when they can never be part of a solution, an empty domain means no solution possible at all → back out of that branch.
- Forward checking is simply sending messages into a variable that just got its value assigned. First step of arc-consistency.

Constraint Propagation Algorithm

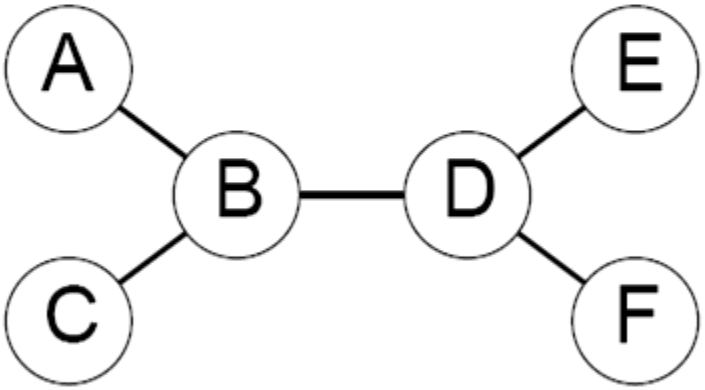
- Maintain all allowed values for each variable.
- At each iteration pick the variable with the fewest remaining values
- For variables with equal # of remaining values, break ties by checking which variable has the largest # of constraints with unassigned variables
- After we picked a variable, tentatively assign it to each of the remaining values in turn and run constraint propagation to convergence.
  (This involves iteratively making all arcs consistent that flow into domains that just have been changed, beginning with the neighbors of the variable you just assigned a value to and iterating until no more changes occur.)
- Among all checked values, pick the one that removed the least values from other domains using constraint propagation.
- Now run constraint propagation once more (or recall it from memory) for the assigned value and remove the the values from the domains of the other variables.
- When domains get empty, back out of that branch.
- Iterate until a solution has been found.
- *(as an alternative you only do constraint propagation after an assignment to prune domains of other variables but avoid doing it for all values. Use simply forward checking with the LCV heuristic to pick a value)*

Can you solve this coloring problem?



Use all heuristics including arc-propagation to solve this problem.

Tree-structured CSPs

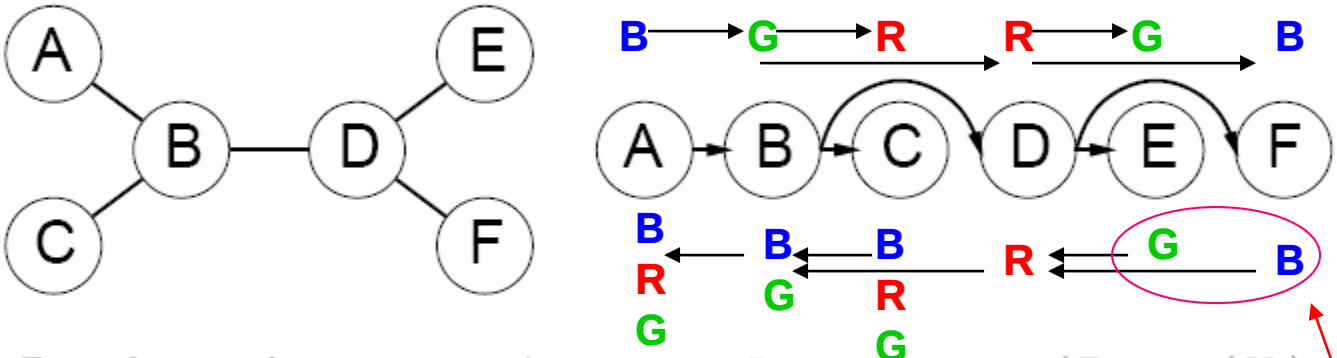


Theorem: if the constraint graph has no loops, the CSP can be solved in $O(n\,d^2)$ time

Compare to general CSPs, where worst-case time is $O(d^n)$

Algorithm for tree-structured CSPs

1. Choose a variable as root, order variables from root to leaves such that every node's parent precedes it in the ordering



2. For j from n down to 2, apply REMOVEINCONSISTENT$(Parent(X_j), X_j)$

3. For j from 1 to n, assign X_j consistently with $Parent(X_j)$

a priori constrained nodes

Note: After the backward pass, there is guaranteed to be a legal choice for a child node for *any* of its leftover values.

This removes any inconsistent values from Parent(Xj), it applies arc-consistency moving backwards.

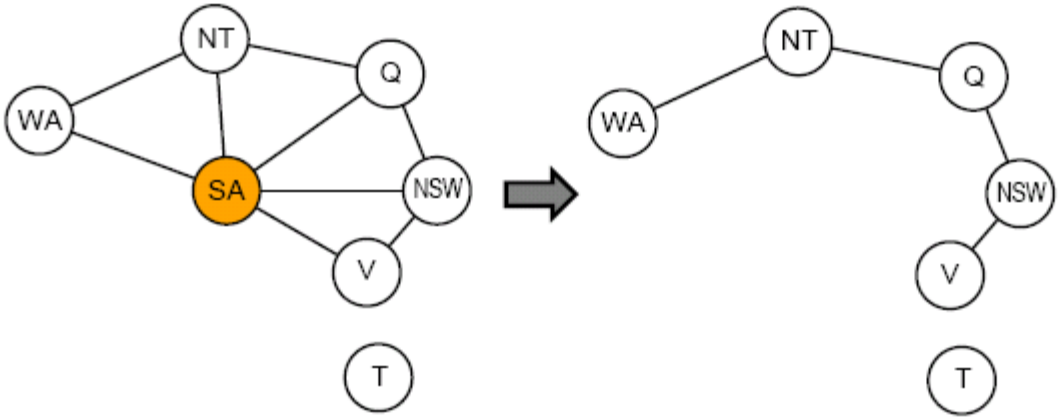
Students: Show that if the domain has two colors, this problem can also be solved.

```
function REMOVE-INCONSISTENT-VALUES(X_i, X_j) returns true iff we remove a value
  removed ← false
  for each x in DOMAIN[X_i] do
    if no value y in DOMAIN[X_j] allows (x,y) to satisfy the constraint between X_i and X_j
      then delete x from DOMAIN[X_i]; removed ← true
  return removed
```

Two methods to use tree structures to solve graph coloring problems. 1

Nearly tree-structured CSPs

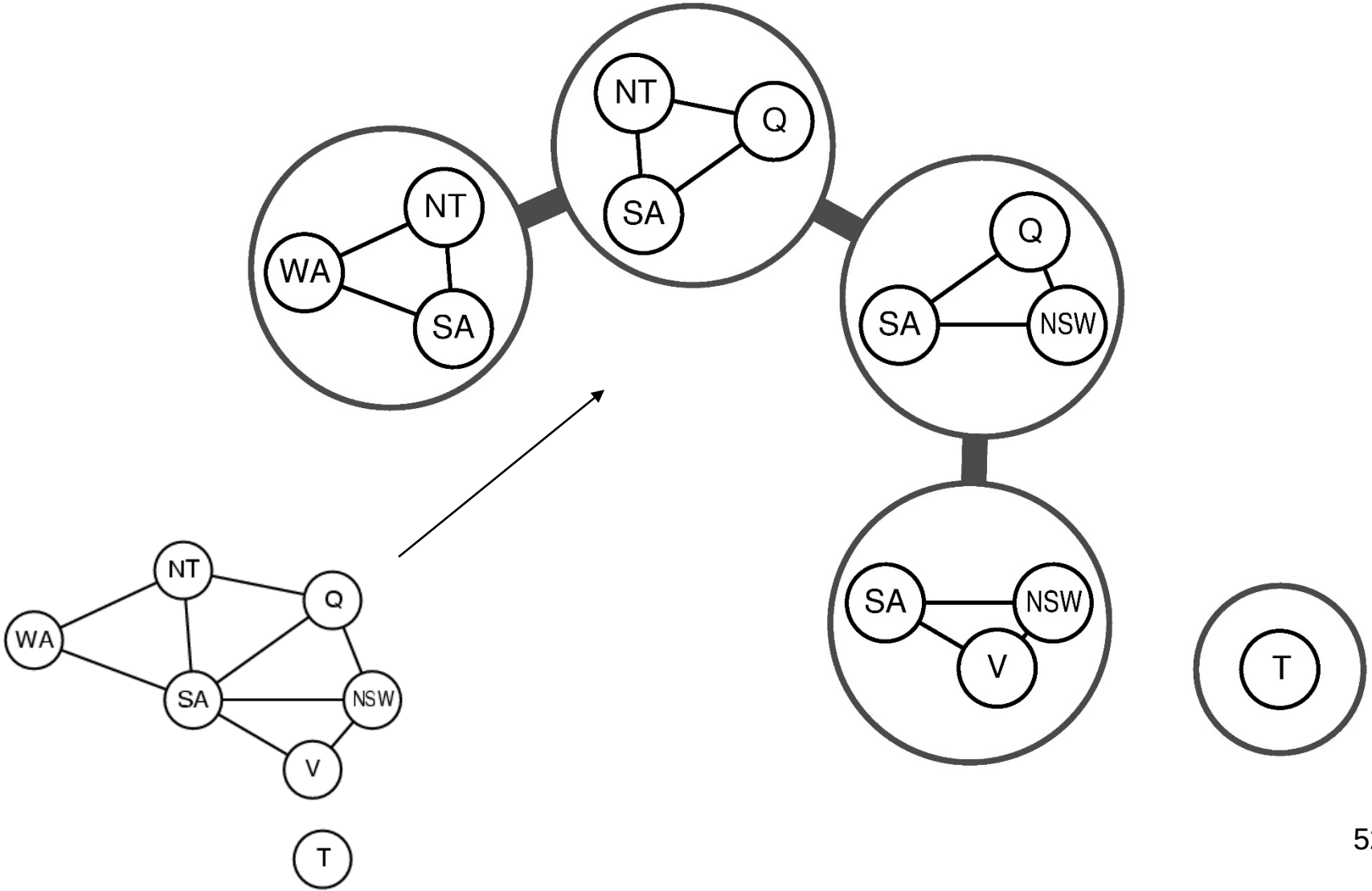
Conditioning: instantiate a variable, prune its neighbors' domains



Cutset conditioning: instantiate (in all ways) a set of variables such that the remaining constraint graph is a tree

Cutset size c $\Rightarrow$ runtime $O(d^c \cdot (n-c)d^2)$, very fast for small c

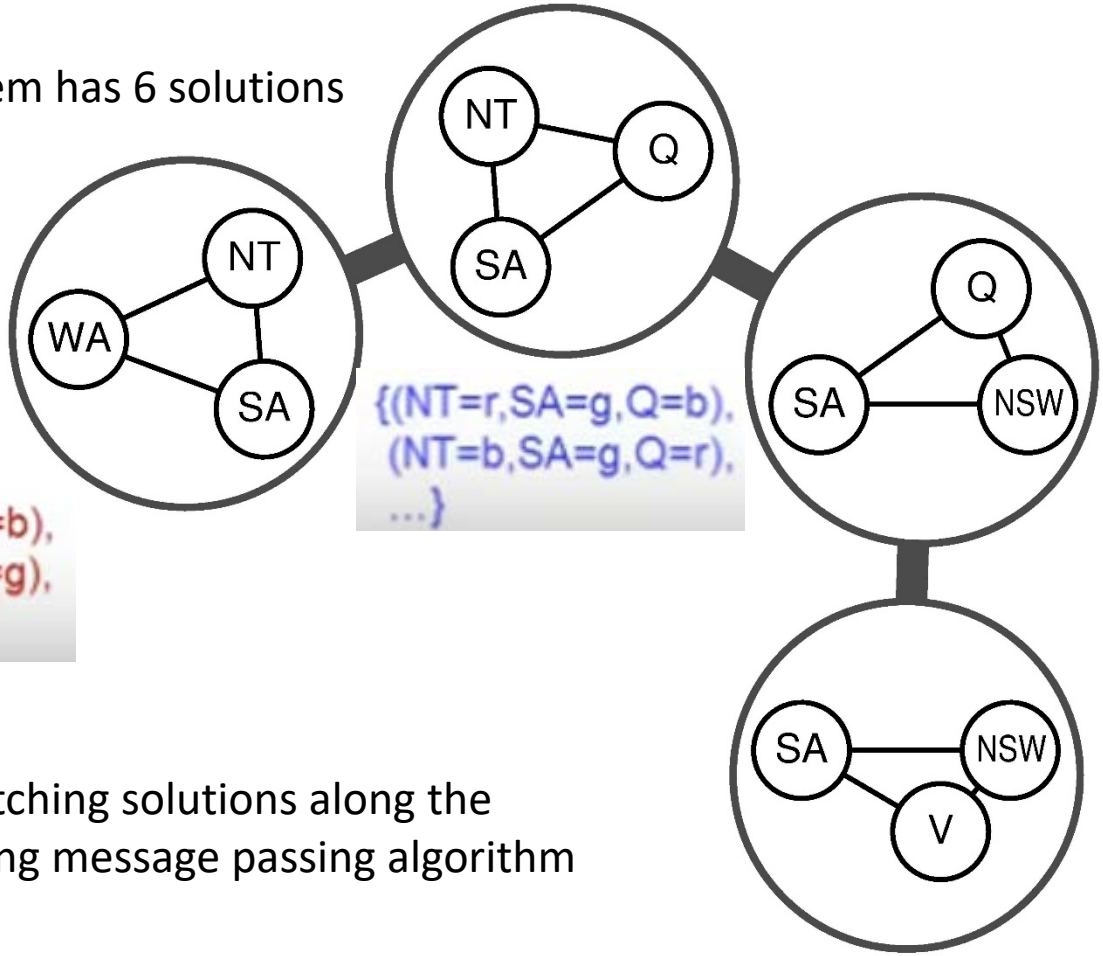
Junction Tree Decompositions



Two methods to use tree structures to solve graph coloring problems

Junction Tree Decompositions

Each subproblem has 6 solutions



Find matching solutions along the tree, using message passing algorithm

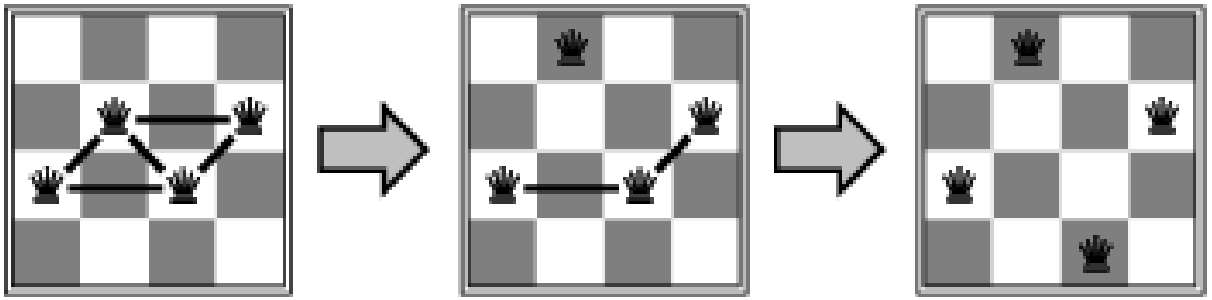
This junction tree decomposition idea is used in many other graph algorithms, such as Bayesian Networks

Min-Conflict Algorithm for CSPs

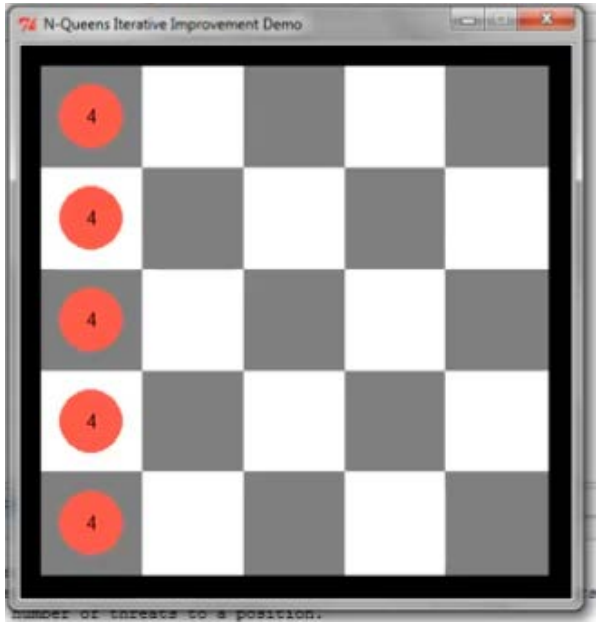
- The particular way we find the solution is unimportant. So we can do local refinements
  - Allow states with some constraints unsatisfied
  - Gradually reassign variable values to reduce the number of unsatisfied constraints.
- Variable selection: randomly select any conflicted variable
- Value selection by min-conflicts heuristic:
  - choose value that violates the fewest constraints
  - i.e., hill-climb with *h(n)* = total number of violated constraints

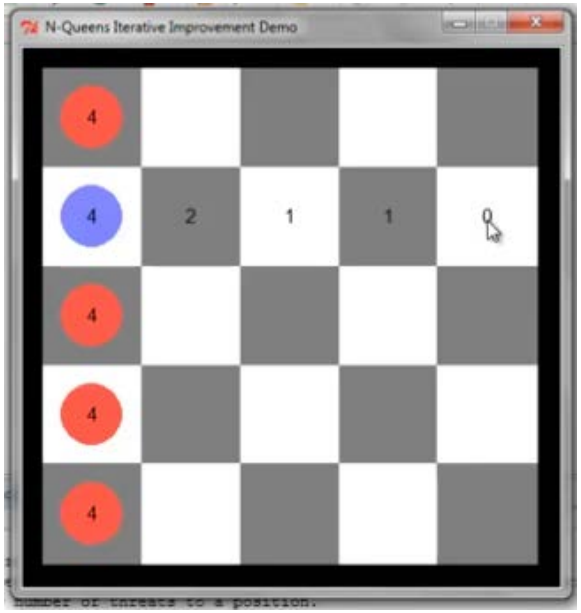
Example: 4-Queens

- States: 4 queens in 4 columns (4^4 = 256 states)
- Actions: move queen in column
- Goal test: no attacks
- Evaluation: *h(n)* = number of attacks

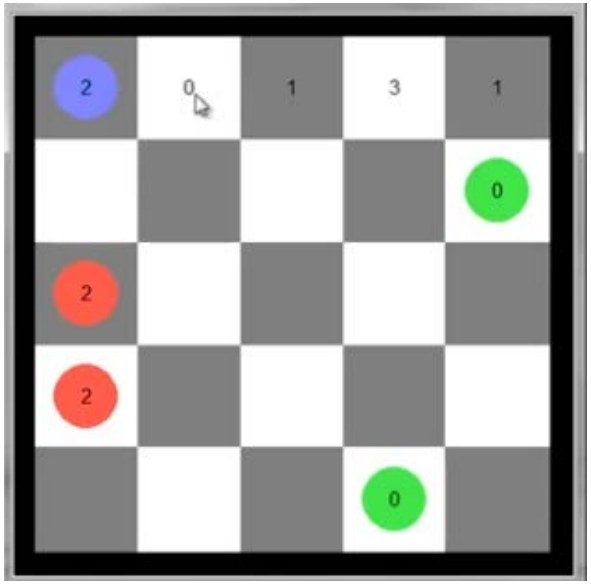


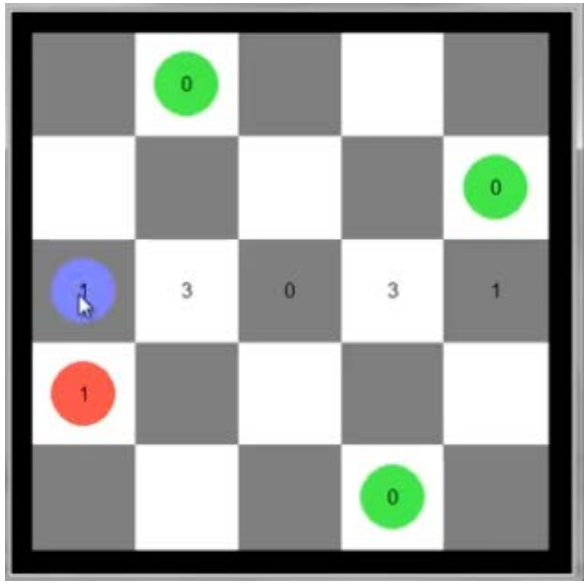
Solve 5-queen problem using minimum conflict repair algorithm

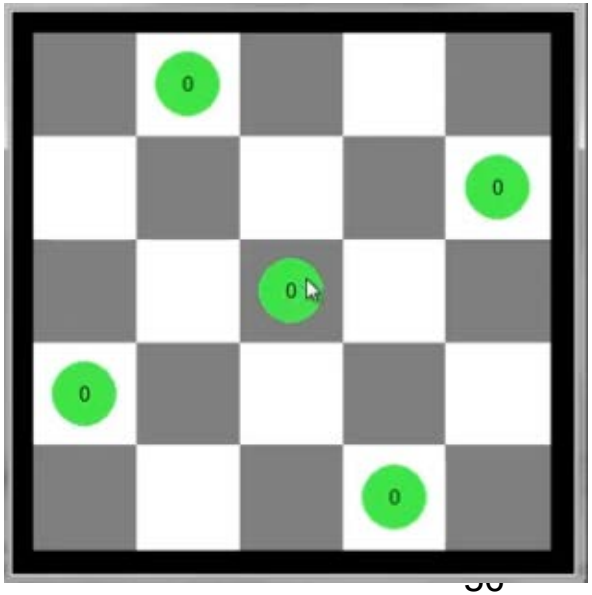




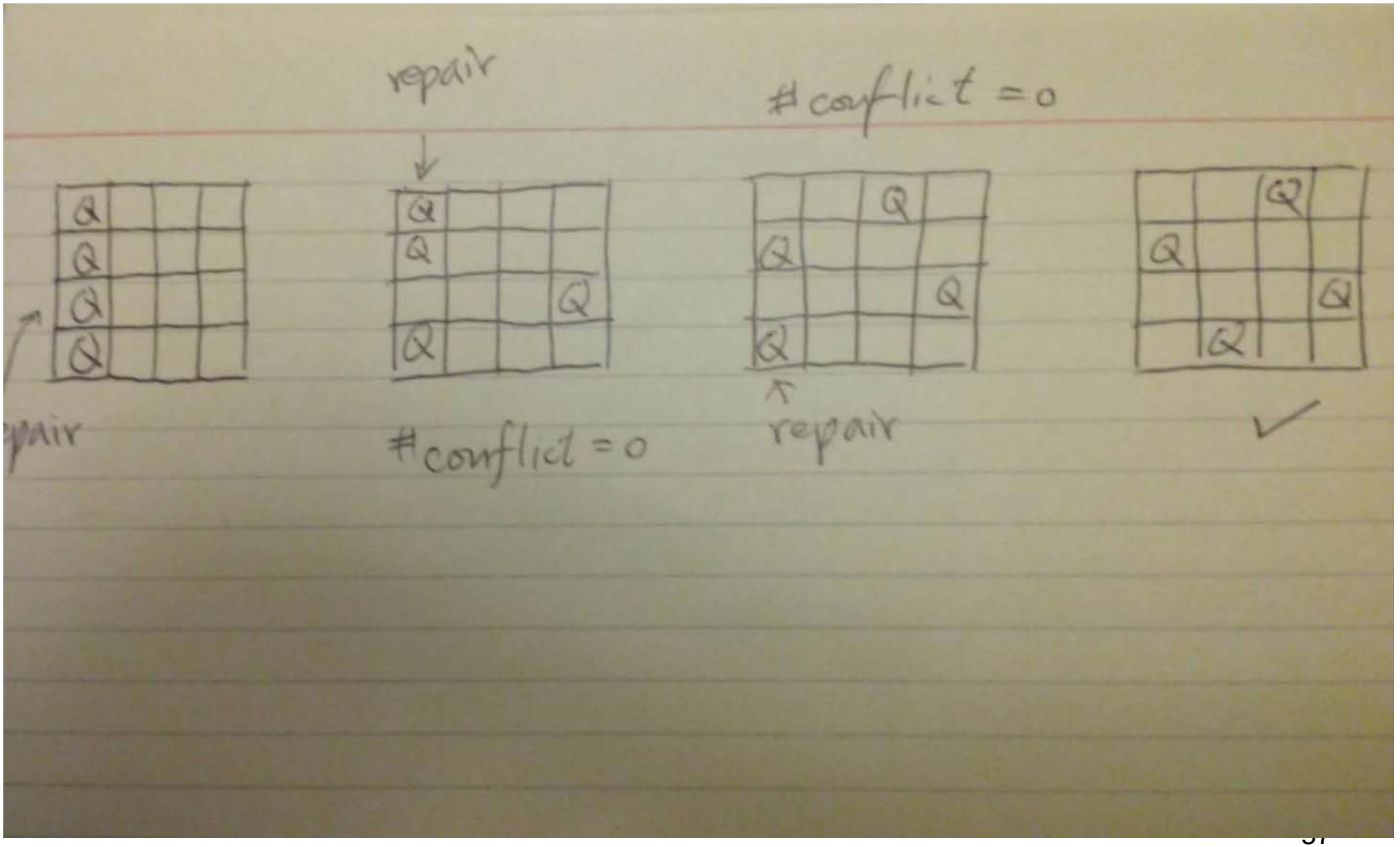








Solve 4-queen problem using
minimum conflict repair algorithm



8-Queen Problem solved using minimum conflict

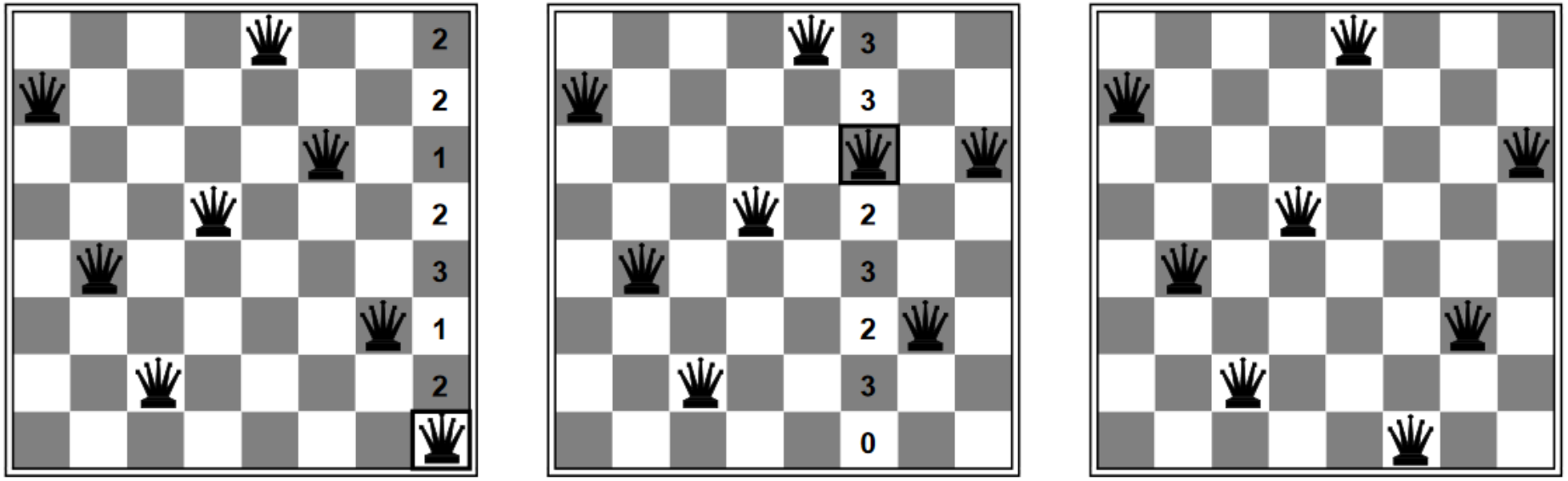


Figure 5.9 A two-step solution for an 8-queens problem using min-conflicts. At each stage, a queen is chosen for reassignment in its column. The number of conflicts (in this case, the number of attacking queens) is shown in each square. The algorithm moves the queen to the min-conflict square, breaking ties randomly.

Homework:
Solve a 6-queen problem using minimum conflict repair algorithm

Number of consistency check (check different color)

Problem	Backtracking	Forward Checking	Min Conflicts
n-queens	> 40,000K	> 40,000K	4K
USA states	> 1,000K	2K	64
Zebra	3,859K	35K	2K

From Russel and Norvig

N-queens: n=2,3,4, ..., 50
USA states map coloring
Zebra puzzle

Zebra Puzzle: Where does zebra live, and who drinks water?

6.7 Consider the following logic puzzle: In five houses, each with a different color, live five persons of different nationalities, each of whom prefers a different brand of candy, a different drink, and a different pet. Given the following facts, the questions to answer are "Where does the zebra live, and in which house do they drink water?"

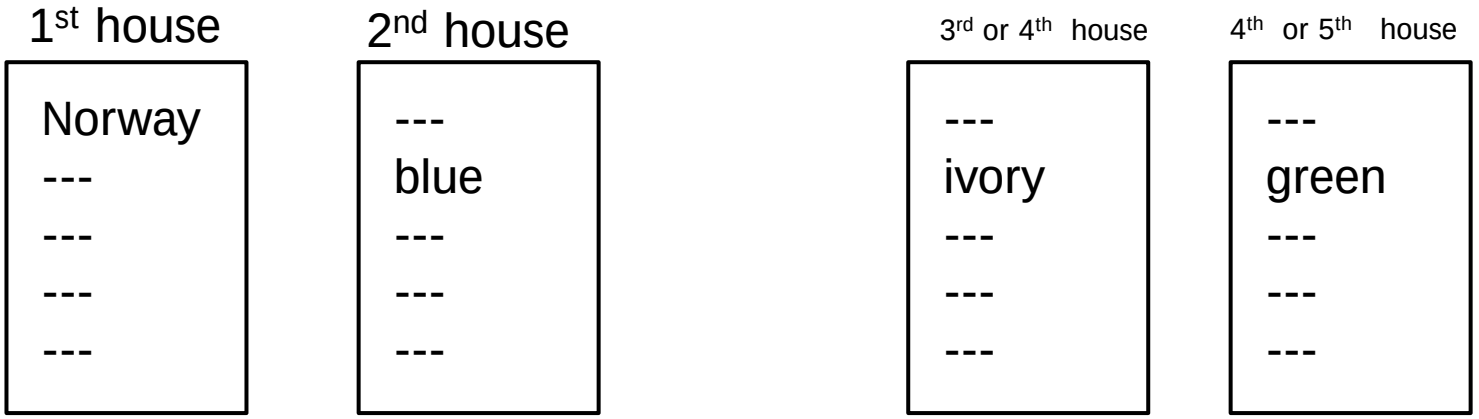
The Englishman lives in the red house.
The Spaniard owns the dog.
The Norwegian lives in the first house on the left.
The green house is immediately to the right of the ivory house.
The man who eats Hershey bars lives in the house next to the man with the fox.
Kit Kats are eaten in the yellow house.
The Norwegian lives next to the blue house.
The Smarties eater owns snails.
The Snickers eater drinks orange juice.
The Ukrainian drinks tea.
The Japanese eats Milky Ways.
Kit Kats are eaten in a house next to the house where the horse is kept.
Coffee is drunk in the green house.
Milk is drunk in the middle house.

Discuss different representations of this problem as a CSP. Why would one prefer one representation over another?

Solve Zebra Puzzle with following clues

House has color domain: blue, green, yellow, ivory, red
Drink: tea, coffee, milk, orange juice, and water (not explicitly named)
Pet: dog, horse, snail, fox, and zebra (not explicitly named)
Candy: snickers, Kit Kats, Hershey bars, smarties, milky ways (invented by authors)

1st major step: get the representation below. Why?

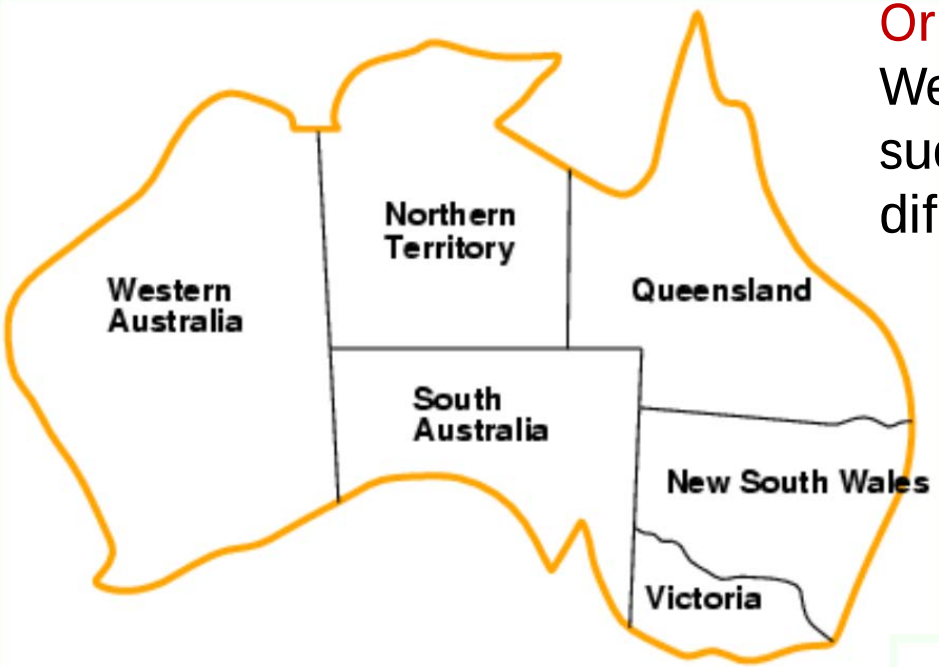


2nd major step: Norwegian's house color is yellow. What are the reasons?

3rd major step: Norwegian drinks water. What are the reasons?

Summary

- CSPs are a special kind of search problem:
  - states defined by values of a fixed set of variables
  - goal test defined by constraints on variable values
- Backtracking = depth-first search with one variable assigned per level.
- Variable ordering and value selection heuristics help significantly
- Forward checking prevents assignments that guarantee later failure
- Constraint propagation (e.g., arc consistency) does additional work to constrain values and detect inconsistencies
- Iterative min-conflicts is usually effective in practice



Original Australia map.

We can use 3 colors to color the map such that neighboring states have different colors.

Suppose Australia map is changed to this. Show that we need 4 colors to color this map.

